Deploy a Container to AWS with GitHub Actions

A complete, beginner-friendly setup guide — build a Docker image in GitHub Actions, store it in Amazon ECR, and run it on an EC2 server. No AWS access keys required.

Who this is for: anyone setting this up in their own AWS account for the first time. · **Time:** 30–45 minutes · **Cost:** free tier eligible (one `t2.micro` / `t3.micro` + a few MB in ECR)

Why this guide exists

This setup has three traps that produce confusing errors. Each one is marked **TRAP** at the exact step where it bites, with the literal error message you would otherwise see. Follow the steps in order and you will not hit any of them.

Contents

1. What you are building — architecture diagram
2. How a deployment actually flows — flow diagram
3. Permissions explained simply — the two robots
4. Before you start — prerequisites
5. Part A — Application files (3 files)
6. Part B — AWS setup (steps 1–8)
7. Part C — GitHub setup (steps 9–11)
8. Part D — Deploy and verify
9. Troubleshooting — every error, and what it really means
10. Final checklist & cleanup

1 · What you are building

You push code to GitHub. GitHub builds a Docker image, stores it in AWS, then tells your EC2 server to download and run it. Your app is then live on the internet. Nothing is deployed by hand, and **no AWS passwords or access keys are ever stored in GitHub**.

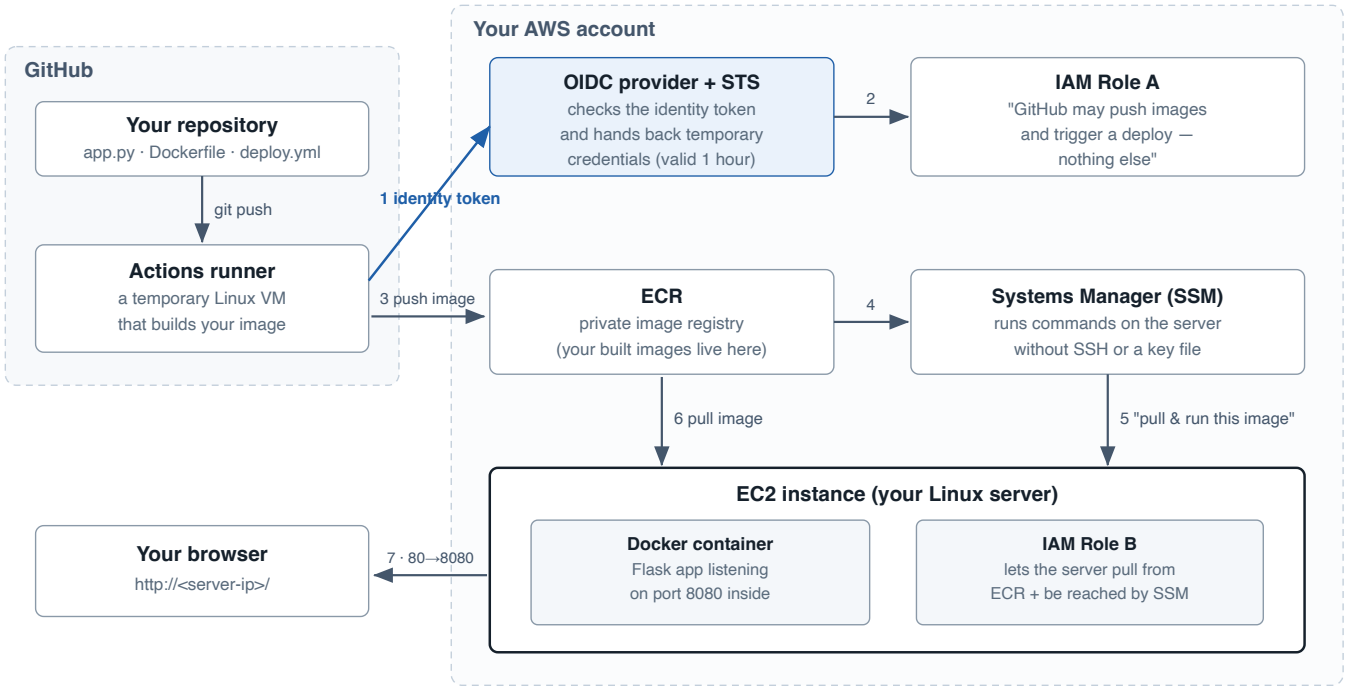


Figure 1 — The whole system. GitHub never holds an AWS key; it proves who it is with a short-lived identity token (step 1) and receives temporary credentials in return.

Piece	What it is, in plain words
ECR	A private warehouse for Docker images, inside your AWS account. Like Docker Hub, but only you can see it.
EC2	A Linux computer that AWS rents you. Your container runs here.
SSM	A remote-control service. It runs commands on your server, so you never need SSH keys in GitHub.
OIDC	The trust handshake that lets GitHub prove its identity to AWS without a stored password.
IAM role	A named set of permissions that something can temporarily borrow.
STS	The AWS service that issues the temporary credentials.

2 · How a deployment actually flows

Every time you run `git push`, these eight things happen in order. If a deploy ever fails, knowing which step it died on tells you exactly where to look.

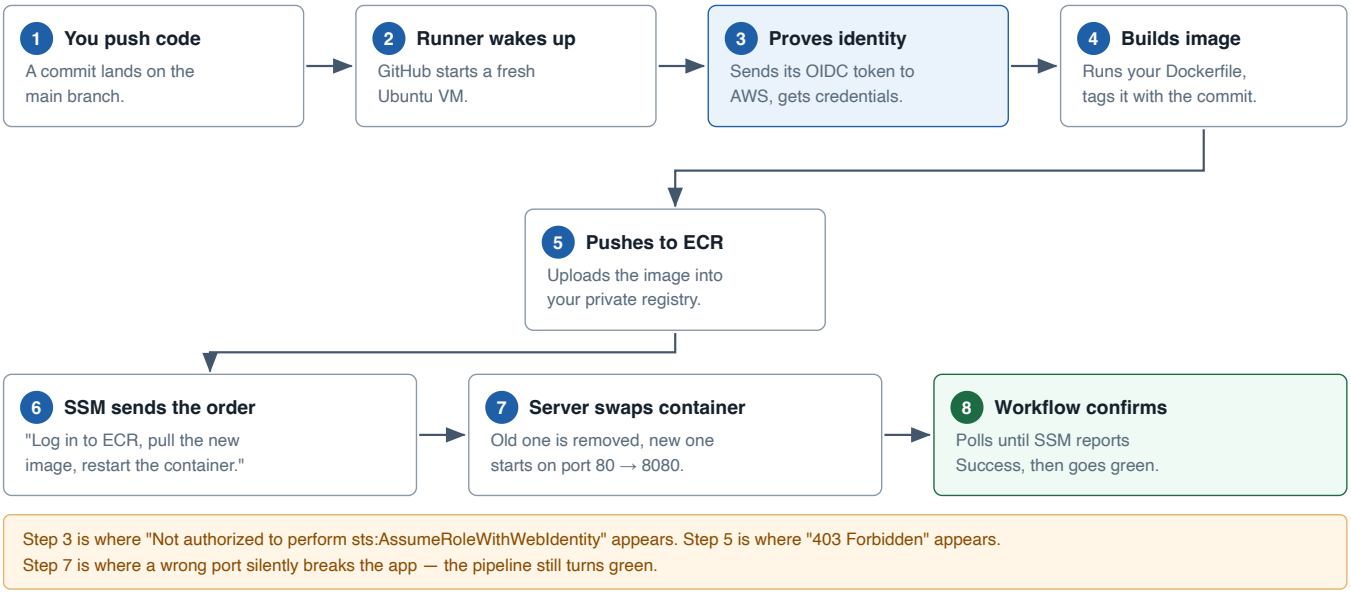


Figure 2 — The eight stages of one deployment, and which stage each classic error belongs to.

3 · Permissions explained simply

This is the part that confuses nearly everyone, so here is the whole idea in one sentence: **there are two separate robots, and each is given only the keys it needs.**

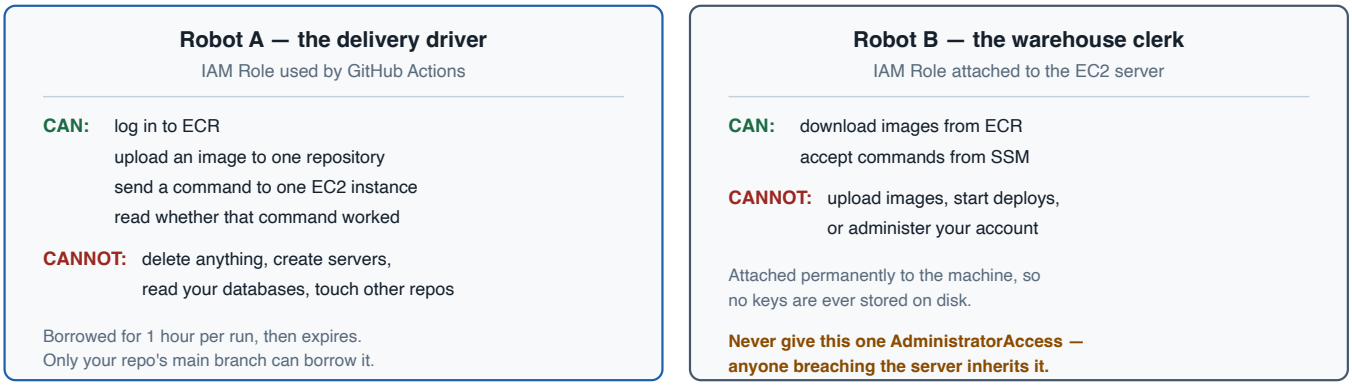


Figure 3 — Two roles, two narrow jobs. Role A pushes and orders; Role B pulls and obeys.

Each role has **two** separate documents, and mixing them up is the single most common source of confusion:

Document	Answers the question	If it is wrong you get
Trust policy who may borrow this role	"Is GitHub's `main` branch in *my* repo allowed to use this role at all?"	`Not authorized to perform sts:AssumeRoleWithWebIdentity`
Permission policy what it may do	"Now that it has the role, may it perform this specific action?"	`AccessDenied` or `403 Forbidden` on a specific call

> **The mental model**
>
> The **trust policy** is the guest list at the door. The **permission policy** is what you are allowed to touch once inside. Getting past the door tells you nothing about what you can do, and being on the guest list under the wrong name means you never get in.

4 · Before you start

You need four things. Verify each one now — it is much cheaper than debugging later.

```
# 1. AWS CLI installed and logged in — must print your account number
aws sts get-caller-identity

# 2. Docker installed (only needed if you want to build locally)
docker --version

# 3. GitHub CLI logged in (optional but makes later steps one-liners)
gh auth status

# 4. A GitHub repository you own, pushed to the main branch
```

Set your variables once

Every command below reuses these. Fill in your own values and paste the whole block into your terminal. **Keep this terminal open** for the rest of the guide.

```
# --- EDIT THESE FOUR LINES ---
export AWS_REGION="ap-south-1"          # your region
export GH_OWNER="your-github-username"  # exact, case-sensitive
export GH_REPO="your-repo-name"         # exact, case-sensitive
export APP_NAME="simple-cicd-app"       # any lowercase name

# --- derived automatically, do not edit ---
export AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export ROLE_NAME="GitHubActions-${APP_NAME}-Role"
export POLICY_NAME="GitHubActions-${APP_NAME}-Policy"
export EC2_ROLE_NAME="EC2-${APP_NAME}-Role"

echo "Account $AWS_ACCOUNT_ID · Region $AWS_REGION · Repo $GH_OWNER/$GH_REPO"
```

5 · Part A — Application files

Three files at the root of your repository. The port number appears in two of them and **must match** — this is Trap 3, so it is worth being deliberate now.

`app.py`

```
from flask import Flask, jsonify
import os

app = Flask(__name__)

# Read the version the pipeline injected. Without this, every deploy looks
# identical in the browser and you cannot tell if a deploy actually landed.
APP_VERSION = os.environ.get("APP_VERSION", "local-dev")

@app.route("/")
def home():
    return jsonify({
        "message": "Hello from AWS EC2!",
        "deployment": "GitHub Actions",
        "version": APP_VERSION,
    })

@app.route("/health")
def health():
    return jsonify({"status": "healthy", "version": APP_VERSION}), 200

if __name__ == "__main__":
    # 0.0.0.0 is required. "localhost" would only be reachable from
    # inside the container, and the app would appear dead from outside.
    app.run(host="0.0.0.0", port=8080)
```

`requirements.txt`

```
Flask==3.1.2
```

`Dockerfile`

```
FROM python:3.12-slim
WORKDIR /app

# Copy requirements first so Docker can cache the install layer.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

# Must equal the port in app.run() above.
EXPOSE 8080
CMD ["python", "app.py"]
```

TRAP 3 — the silent one (we will prevent it in step 11)

Remember the number **8080**. It appears in `app.py`, in the `Dockerfile`, and later in the deploy command as `-p 80:8080`. If the last one disagrees with the first two, **your pipeline turns green and the app is still unreachable** — there is no error message anywhere. It is the hardest failure in this guide to diagnose, because nothing looks broken.

6 · Part B — AWS setup

1 Create the image registry (ECR)

```
aws ecr create-repository \
  --repository-name "$APP_NAME" \
  --region "$AWS_REGION" \
  --image-scanning-configuration scanOnPush=true

# Verify — should print the registry URL
aws ecr describe-repositories --repository-names "$APP_NAME" \
  --region "$AWS_REGION" --query 'repositories[].repositoryUri' --output text
```

2 Teach AWS to trust GitHub (OIDC provider)

This is a one-time setup per AWS account. If you already have one, skip it — a second will fail with `EntityAlreadyExists`, which is harmless.

```
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com \
  --thumbprint-list 6938fd4d98bab03faadb97b34396831e3780aea1

# Verify
aws iam list-open-id-connect-providers
```

> **About that thumbprint**
> AWS no longer validates the thumbprint for GitHub's provider — it verifies against trusted root certificate authorities instead. The value is still a required argument, so the one above is fine. You do not need to hunt for a "current" fingerprint.

3 Find your numeric GitHub IDs — do not skip this

GitHub now embeds permanent numeric IDs in the identity token it sends AWS. You need them for the next step.

```
# With the GitHub CLI
gh api repos/$GH_OWNER/$GH_REPO --jq '{owner_id:.owner.id, repo_id:.id}'

# Or with no tools at all — works in a browser too
curl -s https://api.github.com/repos/$GH_OWNER/$GH_REPO \
  | grep -E '^  "id"|"login"'
```

```
# Save them (substitute the numbers you just got)
export GH_OWNER_ID="169517345"
export GH_REPO_ID="1434296406"
```

TRAP 1 — the error that sends people in circles

If you write the trust policy the way almost every older tutorial shows it, every deploy fails with:

```
Error: Could not assume role with OIDC:
Not authorized to perform sts:AssumeRoleWithWebIdentity
```

You will also see `Assuming role with OIDC` printed a dozen times — that is only the action retrying, not twelve different problems.

Why: GitHub has moved to *immutable identifiers*. The subject it sends is now `repo:owner@1234/repo@5678:ref:refs/heads/main`, not the old `repo:owner/repo:ref:refs/heads/main`. The numeric IDs make renaming a repo or account unable to hijack an existing trust. AWS compares the string exactly, so the old format silently never matches. The policy below accepts **both** spellings, so it works either way and keeps working if GitHub changes the default again.

4 Create the trust policy & Role A

```
cat > trust-policy.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::${AWS_ACCOUNT_ID}:oidc-
provider/token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
        "token.actions.githubusercontent.com:sub": [
          "repo:${GH_OWNER}@${GH_OWNER_ID}/${GH_REPO}@${GH_REPO_ID}:ref:refs/heads/main",
          "repo:${GH_OWNER}/${GH_REPO}:ref:refs/heads/main"
        ]
      }
    }
  }]
}
EOF

aws iam create-role \
  --role-name "$ROLE_NAME" \
  --assume-role-policy-document file://trust-policy.json \
  --description "Lets GitHub Actions deploy $APP_NAME"

# Verify the two subjects are present and spelled correctly
aws iam get-role --role-name "$ROLE_NAME" \
  --query 'Role.AssumeRolePolicyDocument.Statement[0].Condition'
```

Deploying from a branch other than `main`?

Change both `:ref:refs/heads/main` endings to your branch. For tags use `:ref:refs/tags/v1.0.0`. Avoid the tempting wildcard `repo:owner/repo:*` — it would also let pull requests from forks assume your role, which is a real security hole.

5 Create the permission policy for Role A

TRAP 2 — the push that fails at 99%

Grant only the five "obvious" upload actions and every layer uploads successfully, then the push dies at the final step:

```
unknown: unexpected status from HEAD request to
https://....dkr.ecr.../manifests/sha256:3c6e8d...: 403 Forbidden
```

Why: before finalising, Docker sends a `HEAD` request to check whether that manifest already exists. That read maps to `ecr:BatchGetImage` — a *read* permission that a push unexpectedly requires. It is included below.

```
cat > permissions-policy.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ECRLogin",
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*"
    },
    {
      "Sid": "PushImage",
      "Effect": "Allow",
      "Action": [
        "ecr:BatchCheckLayerAvailability",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:CompleteLayerUpload",
        "ecr:PutImage",
        "ecr:BatchGetImage",
        "ecr:GetDownloadUrlForLayer",
        "ecr:DescribeImages"
      ],
      "Resource": "arn:aws:ecr:${AWS_REGION}:${AWS_ACCOUNT_ID}:repository/${APP_NAME}"
    },
    {
      "Sid": "TriggerDeploy",
      "Effect": "Allow",
      "Action": "ssm:SendCommand",
      "Resource": [
        "arn:aws:ssm:${AWS_REGION}::document/AWS-RunShellScript",
        "arn:aws:ec2:${AWS_REGION}:${AWS_ACCOUNT_ID}:instance/*"
      ]
    },
    {
      "Sid": "ReadDeployResult",
      "Effect": "Allow",
      "Action": ["ssm:GetCommandInvocation", "ssm:ListCommandInvocations"],
      "Resource": "*"
    }
  ]
}
EOF

aws iam create-policy \
  --policy-name "$POLICY_NAME" \
  --policy-document file://permissions-policy.json

aws iam attach-role-policy \
  --role-name "$ROLE_NAME" \
  --policy-arn "arn:aws:iam::${AWS_ACCOUNT_ID}:policy/${POLICY_NAME}"
```

Tightening this later

`instance/*` lets the pipeline deploy to any instance in your account. Once your server exists, replace it with that one instance ID (`instance/i-0abc123...`) so a compromised workflow cannot touch other machines.

6 Create Role B for the server

The server itself needs two abilities: **be reachable by SSM**, and **pull from ECR**. Both are covered by AWS-managed policies, so there is nothing to write by hand.

```
cat > ec2-trust.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
EOF

aws iam create-role --role-name "$EC2_ROLE_NAME" \
  --assume-role-policy-document file://ec2-trust.json

# Required for SSM to see the server at all
aws iam attach-role-policy --role-name "$EC2_ROLE_NAME" \
  --policy-arn arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore

# Read-only ECR access: enough to pull, cannot overwrite your images
aws iam attach-role-policy --role-name "$EC2_ROLE_NAME" \
  --policy-arn arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly

# Wrap the role in an instance profile — EC2 cannot use a bare role
aws iam create-instance-profile --instance-profile-name "$EC2_ROLE_NAME"
aws iam add-role-to-instance-profile \
  --instance-profile-name "$EC2_ROLE_NAME" --role-name "$EC2_ROLE_NAME"
```

> **Resist the shortcut**
>
> When SSM "cannot find" a server, the quickest-looking fix is to attach `AdministratorAccess`. Do not. Anyone who gets code execution on that box — through your app, a dependency, an exposed port — instantly inherits full control of your AWS account. `AmazonSSMManagedInstanceCore` is what SSM actually needs.

7 Create the firewall rule (security group)

```
export VPC_ID=$(aws ec2 describe-vpcs --region "$AWS_REGION" \
  --filters "Name=isDefault,Values=true" --query 'Vpcs[0].VpcId' --output text)

export SG_ID=$(aws ec2 create-security-group \
  --group-name "${APP_NAME}-sg" \
  --description "Web access for $APP_NAME" \
  --vpc-id "$VPC_ID" --region "$AWS_REGION" \
  --query 'GroupId' --output text)

# Allow web traffic from anywhere
aws ec2 authorize-security-group-ingress --group-id "$SG_ID" \
  --protocol tcp --port 80 --cidr 0.0.0.0/0 --region "$AWS_REGION"

echo "Security group: $SG_ID"
```

You do **not** need to open port 22. SSM replaces SSH entirely, which is one of the nicest properties of this setup — no key files to lose or leak.

8 Launch the server

The start-up script below installs Docker *and* the AWS CLI. The CLI is easy to forget and the deploy needs it on the server, because the pull command runs `aws ecr get-login-password` there.

```
cat > user-data.sh <<'EOF'
#!/bin/bash
set -eux
apt-get update -y
apt-get install -y docker.io unzip curl
systemctl enable --now docker

# AWS CLI v2 — required on the server to log in to ECR
curl -s "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o /tmp/awscli.zip
unzip -q /tmp/awscli.zip -d /tmp && /tmp/aws/install

# SSM agent ships with Ubuntu images; make sure it is running
snap start amazon-ssm-agent 2>/dev/null || systemctl enable --now amazon-ssm-agent || true
EOF

# Latest Ubuntu 22.04 AMI for your region, looked up automatically
export AMI_ID=$(aws ec2 describe-images --region "$AWS_REGION" \
  --owners 099720109477 \
  --filters "Name=name,Values=ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*" \
  --query 'sort_by(Images,&CreationDate)[-1].ImageId' --output text)

export EC2_INSTANCE_ID=$(aws ec2 run-instances --region "$AWS_REGION" \
  --image-id "$AMI_ID" \
  --instance-type t3.micro \
  --iam-instance-profile Name="$EC2_ROLE_NAME" \
  --security-group-ids "$SG_ID" \
  --user-data file://user-data.sh \
  --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=$APP_NAME}]" \
  --query 'Instances[0].InstanceId' --output text)

echo "Instance: $EC2_INSTANCE_ID"
```

Wait for SSM before continuing

The start-up script takes 2–4 minutes. Until it finishes the server is invisible to SSM and your deploy would fail with `InvalidInstanceId`. Poll until this prints `Online`:

```
aws ssm describe-instance-information --region "$AWS_REGION" \
  --filters "Key=InstanceIds,Values=$EC2_INSTANCE_ID" \
  --query 'InstanceInformationList[].PingStatus' --output text
```

Give the server a permanent address (recommended)

By default the public IP changes whenever the instance is stopped and started, which quietly breaks any bookmark or DNS record. An Elastic IP pins it for free while attached.

```
export EIP_ALLOC=$(aws ec2 allocate-address --domain vpc \
  --region "$AWS_REGION" --query 'AllocationId' --output text)

aws ec2 associate-address --instance-id "$EC2_INSTANCE_ID" \
  --allocation-id "$EIP_ALLOC" --region "$AWS_REGION"

aws ec2 describe-instances --instance-ids "$EC2_INSTANCE_ID" \
  --region "$AWS_REGION" \
  --query 'Reservations[].Instances[].PublicIpAddress' --output text
```

7 · Part C — GitHub setup

9 Store the four variables

These are **variables**, not secrets — none of them is sensitive. A role ARN and an account ID are useless to an attacker without the trust policy naming their repo.

```
gh variable set AWS_REGION       --body "$AWS_REGION"       --repo "$GH_OWNER/$GH_REPO"
gh variable set AWS_ACCOUNT_ID   --body "$AWS_ACCOUNT_ID"   --repo "$GH_OWNER/$GH_REPO"
gh variable set EC2_INSTANCE_ID  --body "$EC2_INSTANCE_ID"  --repo "$GH_OWNER/$GH_REPO"
gh variable set AWS_ROLE_ARN \
  --body "arn:aws:iam::${AWS_ACCOUNT_ID}:role/${ROLE_NAME}" --repo "$GH_OWNER/$GH_REPO"

# Verify all four
gh variable list --repo "$GH_OWNER/$GH_REPO"
```

By hand instead: **Repository → Settings → Secrets and variables → Actions → Variables** tab → *New repository variable*.

Variable	Example value
`AWS_REGION`	`ap-south-1`
`AWS_ACCOUNT_ID`	`123456789012`
`EC2_INSTANCE_ID`	`i-0abc123def456789`
`AWS_ROLE_ARN`	`arn:aws:iam::123456789012:role/GitHubActions-simple-cicd-app-Role`

10 Add the workflow

Save as `.github/workflows/deploy.yml`. Note `-p 80:8080` near the bottom — that is Trap 3 neutralised.

```yaml
name: Build and Deploy to EC2

on:
  push:
    branches: [main]
  workflow_dispatch:        # adds a manual "Run workflow" button

permissions:
  contents: read
  id-token: write          # REQUIRED: lets the job request an OIDC token

concurrency:
  group: ${{ github.workflow }}-production
  cancel-in-progress: false   # never interrupt a half-finished deploy

jobs:
  deploy:
    runs-on: ubuntu-latest
    env:
      AWS_REGION: ${{ vars.AWS_REGION }}
      AWS_ACCOUNT_ID: ${{ vars.AWS_ACCOUNT_ID }}
      EC2_INSTANCE_ID: ${{ vars.EC2_INSTANCE_ID }}
      ECR_REPOSITORY: simple-cicd-app    # must equal $APP_NAME

    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Configure AWS credentials using OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ vars.AWS_ROLE_ARN }}
          aws-region: ${{ vars.AWS_REGION }}

      - name: Login to Amazon ECR
        id: ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build and push Docker image
        env:
          ECR_REGISTRY: ${{ steps.ecr.outputs.registry }}
          IMAGE_TAG: ${{ github.sha }}
        run: |
          set -euo pipefail
          IMAGE_URI="${ECR_REGISTRY}/${ECR_REPOSITORY}:${IMAGE_TAG}"
          docker build --pull -t "$IMAGE_URI" .
          docker push "$IMAGE_URI"
          echo "IMAGE_URI=$IMAGE_URI" >> "$GITHUB_ENV"
          echo "IMAGE_TAG=$IMAGE_TAG" >> "$GITHUB_ENV"

      - name: Deploy image to EC2 through SSM
        run: |
          set -euo pipefail
          REGISTRY="${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com"

          # jq builds the JSON so quotes and spaces can never break it
          PARAMS=$(jq -n \
            --arg region "$AWS_REGION" \
            --arg registry "$REGISTRY" \
            --arg image "$IMAGE_URI" \
            --arg version "$IMAGE_TAG" \
```

```
    '{
      commands: [
        "set -eu",
        ("aws ecr get-login-password --region " + $region +
         " | sudo docker login --username AWS --password-stdin " + $registry),
        ("sudo docker pull " + $image),
        "sudo docker rm -f simple-cicd-app || true",
        ("sudo docker run -d --restart unless-stopped " +
         "--name simple-cicd-app -p 80:8080 " +
         "-e APP_VERSION=" + $version + " " + $image)
      ]
    }')

  COMMAND_ID=$(aws ssm send-command \
    --instance-ids "$EC2_INSTANCE_ID" \
    --document-name "AWS-RunShellScript" \
    --parameters "$PARAMS" \
    --query "Command.CommandId" --output text)
  echo "SSM command: $COMMAND_ID"

  # Poll until the server reports back, up to 5 minutes
  for i in $(seq 1 60); do
    STATUS=$(aws ssm get-command-invocation \
      --command-id "$COMMAND_ID" --instance-id "$EC2_INSTANCE_ID" \
      --query "Status" --output text 2>/dev/null || echo "Pending")
    echo "Status: $STATUS"
    case "$STATUS" in
      Success) echo "Deployed."; exit 0 ;;
      Failed|Cancelled|TimedOut)
        aws ssm get-command-invocation --command-id "$COMMAND_ID" \
          --instance-id "$EC2_INSTANCE_ID" --output json || true
        exit 1 ;;
    esac
    sleep 5
  done
  echo "Timed out."; exit 1
```

> **Why `id-token: write` matters**
>
> Omit it and the job cannot request an identity token at all. The error (`Credentials could not be loaded`) looks like an AWS problem, but the cause is entirely on the GitHub side. It must sit at the top level or inside the job — not under `steps`.

11 Confirm the port numbers agree

Thirty seconds now saves the worst debugging session in this guide. All three must be **8080**:

File	Line	Port
`app.py`	`app.run(host="0.0.0.0", port=8080)`	8080
`Dockerfile`	`EXPOSE 8080`	8080
`deploy.yml`	`-p 80:8080`	8080

The `80` on the left of the colon is the public web port and should stay 80. Only the right-hand number must match your app.

8 · Part D — Deploy and verify

```
git add .
git commit -m "Add CI/CD pipeline"
git push origin main

# Watch it run live
gh run watch --exit-status
```

All six steps should pass in roughly 30–60 seconds:

```
✓ Checkout source code
✓ Configure AWS credentials using OIDC
✓ Login to Amazon ECR
✓ Build and push Docker image
✓ Deploy image to EC2 through SSM
✓ Complete job
```

Check it in your browser

```
aws ec2 describe-instances --instance-ids "$EC2_INSTANCE_ID" \
  --region "$AWS_REGION" \
  --query 'Reservations[].Instances[].PublicIpAddress' --output text
```

Open `http://<that-ip>/` and you should see:

```
{"deployment":"GitHub Actions","message":"Hello from AWS EC2!","version":"a1b2c3d..."}
```

Type `http://` explicitly

There is no TLS certificate on this server. Modern browsers silently upgrade a bare IP to `https://`, which fails with a connection or certificate warning and makes a perfectly healthy app look dead. If you see an SSL error, check the address bar — you are on `https`. Testing with `curl` avoids the ambiguity entirely:

```
curl -i http://<that-ip>/
```

The version field is your deploy receipt

`version` shows the exact commit SHA that is running. Push a change and watch it update — that is how you know a deploy truly landed, rather than trusting a green checkmark. If the SHA does not change, the container never actually restarted.

9 · Troubleshooting

Error you see	What it really means	Fix
`Not authorized to perform sts:AssumeRoleWithWebIdentity`	The `sub` in the trust policy does not match what GitHub sent — usually the immutable-ID format, a wrong branch, or different capitalisation of owner/repo.	Step 4. Compare the real value against your policy using the CloudTrail command below.
`403 Forbidden` on a `manifests/sha256:…` HEAD request	`ecr:BatchGetImage` is missing. Layers upload fine; only the final manifest check fails.	Step 5. Add the three read actions, then re-run.
Pipeline all green, browser shows nothing	Port mismatch. Docker is forwarding port 80 to a container port nothing listens on.	Step 11. Confirm with `docker ps` — see command below.
`Credentials could not be loaded`	`id-token: write` is missing from the workflow.	Step 10.
`InvalidInstanceId`	SSM cannot see the server: still booting, missing `AmazonSSMManagedInstanceCore`, or the agent is not running.	Steps 6 & 8. Wait for `Online`.
`EntityAlreadyExists`	Harmless — that role, policy, or OIDC provider already exists.	Skip the step, or delete the old one first.
`repository does not exist` during push	`ECR_REPOSITORY` in the workflow does not match the real repository name.	Steps 1 & 10 — the names must be identical.
Deploy step fails with `aws: command not found`	The AWS CLI is not installed *on the server*, so it cannot log in to ECR.	Step 8 — re-run the CLI install lines on the instance.
Connection times out, no response at all	Security group is not allowing port 80 inbound.	Step 7.

The two commands that answer almost everything

What subject did GitHub actually send? This ends all trust-policy guesswork — it prints the exact string AWS received, which you can paste straight into your policy.

```
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=AssumeRoleWithWebIdentity \
  --region "$AWS_REGION" --max-results 3 \
  --query 'Events[].Username' --output text
```

```
# Example output — note the numeric IDs the old format omits
repo:your-name@169517345/your-repo@1434296406:ref:refs/heads/main
```

Is the container running, and on which ports? Reads the truth from the server without SSH.

```
CID=$(aws ssm send-command --instance-ids "$EC2_INSTANCE_ID" \
  --region "$AWS_REGION" --document-name "AWS-RunShellScript" \
  --parameters 'commands=["sudo docker ps -a --format \"{{.Names}} {{.Status}} {{.Ports}}\"","sudo
docker logs --tail 20 simple-cicd-app"]' \
  --query "Command.CommandId" --output text)

sleep 6
aws ssm get-command-invocation --command-id "$CID" \
  --instance-id "$EC2_INSTANCE_ID" --region "$AWS_REGION" \
  --query 'StandardOutputContent' --output text
```

```
# Healthy — port 80 maps to 8080:
simple-cicd-app Up 2 minutes 0.0.0.0:80->8080/tcp

# Broken — forwarding to 5000, where nothing listens:
simple-cicd-app Up 2 minutes 8080/tcp, 0.0.0.0:80->5000/tcp
```

10 · Final checklist

✓	Item	Quick verification
☐	ECR repository exists	`aws ecr describe-repositories --repository-names $APP_NAME`
☐	OIDC provider registered	`aws iam list-open-id-connect-providers`
☐	Trust policy has **both** subject formats	`aws iam get-role --role-name $ROLE_NAME`
☐	Permission policy includes `ecr:BatchGetImage`	Check the `PushImage` statement
☐	EC2 role has `AmazonSSMManagedInstanceCore`	`aws iam list-attached-role-policies --role-name $EC2_ROLE_NAME`
☐	EC2 role does *not* have `AdministratorAccess`	Same command as above
☐	Security group allows port 80	`aws ec2 describe-security-groups --group-ids $SG_ID`
☐	Server shows `Online` in SSM	`aws ssm describe-instance-information`
☐	All four GitHub variables set	`gh variable list`
☐	Port is 8080 in all three files	Step 11 table
☐	Browser shows the commit SHA as `version`	`curl http://<ip>/`

Cleanup — delete everything

Run in this order; dependencies block out-of-order deletes.

```
aws ec2 terminate-instances --instance-ids "$EC2_INSTANCE_ID" --region "$AWS_REGION"
aws ec2 release-address --allocation-id "$EIP_ALLOC" --region "$AWS_REGION"
aws ecr delete-repository --repository-name "$APP_NAME" --force --region "$AWS_REGION"

aws iam detach-role-policy --role-name "$ROLE_NAME" \
  --policy-arn "arn:aws:iam::${AWS_ACCOUNT_ID}:policy/${POLICY_NAME}"
aws iam delete-policy --policy-arn "arn:aws:iam::${AWS_ACCOUNT_ID}:policy/${POLICY_NAME}"
aws iam delete-role --role-name "$ROLE_NAME"

aws iam remove-role-from-instance-profile \
  --instance-profile-name "$EC2_ROLE_NAME" --role-name "$EC2_ROLE_NAME"
aws iam delete-instance-profile --instance-profile-name "$EC2_ROLE_NAME"
aws iam detach-role-policy --role-name "$EC2_ROLE_NAME" \
  --policy-arn arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore
aws iam detach-role-policy --role-name "$EC2_ROLE_NAME" \
  --policy-arn arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly
aws iam delete-role --role-name "$EC2_ROLE_NAME"

# Security group must come after the instance is fully terminated
aws ec2 delete-security-group --group-id "$SG_ID" --region "$AWS_REGION"
```

Sensible next steps

- **Add HTTPS** — put an Application Load Balancer with an ACM certificate in front, or run Caddy on the instance for automatic Let's Encrypt certificates.
- **Narrow the SSM permission** — replace `instance/*` in Role A with your single instance ID.
- **Add a health check gate** — have the workflow `curl /health` after deploying and fail if it does not return 200, so a broken image cannot sit there unnoticed.
- **Keep the previous image** — tag each build `latest` as well, so rolling back is one `docker run` with the prior SHA.
- **Stop paying for old images** — add an ECR lifecycle policy to expire untagged images after a few days.

The three traps, one last time

1. Trust policy must use GitHub's immutable-ID subject format (step 4). **2.** ECR push needs the read action `ecr:BatchGetImage` (step 5). **3.** The container port in `-p 80:8080` must match the port your app listens on (step 11) — this one fails silently.